МИНИСТЕРСТВО ОБРАЗОВАНИЯ СВЕРДЛОВСКОЙ ОБЛАСТИ
ГАПОУ СО «ЕКАТЕРИНБУРГСКИЙ МОНТАЖНЫЙ КОЛЛЕДЖ»

КУРСОВОЙ ПРОЕКТ
ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
Первый этап: REST-сервис на FastAPI

По дисциплине «МДК 05.03 Тестирование информационных систем»

Тема: «Создание и тестирование АРМ для обработки информации об участии студентов в олимпиадном движении, конкурсах профессионального мастерства»

09 02 07 — КП — _____

Выполнил: Пеннер Андрей Владимирович

Группа: _____

Номер студенческого: _____

Руководитель курсового проекта: _____

Подписи: _____

Оценка: _____

Екатеринбург
2026

ВЕДОМОСТЬ ПРОЕКТА

Таблица 1 — Состав комплекта первого этапа

№ стр.	Формат	Обозначение	Наименование	Кол. листов	№ экз.	Примечание
1	A4	ПЗ	Пояснительная записка	60	1	DOCX / PDF
2	—	ИС	REST-сервис API первого этапа	—	1	app/
3	—	—	Руководства API и установки	—	1	docs/API.md; INSTALL.md
4	—	—	План, тест-кейсы, отчёт	—	1	docs/TEST_*.md
5	—	—	Автоматические тесты	—	1	tests/
6	—	—	Зависимости и настройка БД	—	1	requirements*.txt; database.sql; init_db.py

Комплект содержит исходный REST-сервис, тесты, зависимости и документацию первого этапа. Конфигурация .env, виртуальное окружение и рабочие пользовательские файлы в Git не включаются. Учебные записи и документы создаются командой python init_db.py только в пустой базе. Состав комплекта приведён в таблице 1.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ · · · 4
1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ · · · 5
1.1 Уточнения и критерии приёмки · · · 6
2 АНАЛИТИЧЕСКАЯ ЧАСТЬ · · · 7
2.1 Цели, функции и задачи системы · · · 7
2.2 Описание предметной области · · · 8
2.3 Пользователи и разграничение доступа · · · 9
2.4 Внутренняя архитектура · · · 11
2.5 Проектные решения по тестированию · · · 12
3 ПРОЕКТНАЯ ЧАСТЬ · · · 25
3.1 Стек технологий · · · 25
3.2 Модель и ограничения базы данных · · · 26
3.2.1 Таблица users · · · 28
3.2.2 Таблица olympiads · · · 29
3.2.3 Таблица participations · · · 30
3.2.4 Таблица attachments · · · 30
3.2.5 Связи и ограничения · · · 32
3.3 Файловая структура и назначение модулей · · · 33
3.4 Алгоритм записи на олимпиаду · · · 36
3.4.1 Подготовка, отзыв и публикация протокола · · · 38
3.4.2 Аутентификация и отзыв токенов · · · 39
3.5 Документация REST API · · · 41
3.5.1 Примеры запросов и обработка ответа · · · 43
3.5.2 Работа пользователей с API · · · 45
3.5.3 Руководство по развёртыванию · · · 46
3.6 Результаты тестирования · · · 48
4 ГЛОССАРИЙ · · · 50
5 СПИСОК ИСТОЧНИКОВ · · · 52
ЗАКЛЮЧЕНИЕ · · · 54
Приложение 1 — Баг-репорт · · · 55
Приложение 2 — Примеры работы API · · · 57

ВВЕДЕНИЕ

Тема работы — создание и тестирование АРМ для обработки информации об участии студентов в олимпиадном движении и конкурсах профессионального мастерства. Для каждого мероприятия требуется хранить положение, даты, список участников, баллы и результаты. Раздельное ведение таблиц и документов приводит к повторному вводу и расхождениям при подготовке отчётов.

Актуальность проекта связана с необходимостью упорядочить эти сведения и разграничить ответственность пользователей. Студенту нужен доступ к своим заявкам и результатам, преподавателю — к подготовке документов своего мероприятия, администратору — к учётным записям и публикации проверенных протоколов. Гостю достаточно открытых сведений без доступа к редактированию и личным данным учётных записей.

Объект разработки — обработка сведений об участии студентов в олимпиадном движении и конкурсах профессионального мастерства. Предмет разработки — REST-сервис, который хранит эти сведения в PostgreSQL и предоставляет операции через HTTP. Основанием служит персональное техническое задание, выданное Пеннер Андрею Владимировичу [25].

Цель первого этапа — разработать и проверить самостоятельный REST-сервис на FastAPI. Для достижения цели требуется определить сущности и права пользователей; реализовать обработку заявок, документов и результатов; подготовить отчёты Word и Excel; проверить положительные и отрицательные сценарии; составить документацию по установке и использованию.

Работа ограничена REST API первого этапа со сроком до 15 декабря 2026 года. Django и его пользовательские страницы относятся ко второму этапу и здесь не разрабатываются. В записке рассматриваются задание, предметная область, устройство API, испытания и установка. Основание и оформление определены предоставленными документами [22, 23, 24, 25].

Задачи первого этапа:

1) проанализировать сведения об олимпиадах, участниках и результатах и определить модель данных;

2) реализовать REST API, разграничение доступа, заявки, публикацию и отчёты;

3) подготовить положительные и отрицательные тесты, проверить их на отдельной PostgreSQL и измерить покрытие;

4) описать установку, работу с API и фактические результаты испытаний.

1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Полная тема проекта: «Создание и тестирование АРМ для обработки информации об участии студентов в олимпиадном движении, конкурсах профессионального мастерства». Разрабатываемый сервис предназначен для ввода, хранения и обработки сведений об олимпиадах, участниках и результатах. Он должен обеспечивать систематизацию информации и формирование отчётности.

Гость просматривает список олимпиад, отбирает их по году, получает карточку, положение, задания прошлых лет и опубликованные протоколы. Студент входит в систему, изменяет собственный профиль, подаёт заявки на несколько мероприятий и просматривает свои опубликованные результаты.

Администратор создаёт преподавателей, редактирует существующие учётные записи, заполняет карточки мероприятий, назначает ответственных и публикует подготовленные протоколы. Ему доступны протокол выбранной олимпиады и годовой отчёт. Преподаватель создаёт студентов, ведёт документы и результаты своей олимпиады, готовит протокол и при необходимости отзывает его до публикации.

В карточке студента хранятся ФИО, дата рождения, электронная почта, логин, учебное заведение, уровень образования, курс и специальность. Пароль принимается при создании, но в открытом виде не сохраняется. Для мероприятия задаются название, описание, ответственный преподаватель и даты. В заявке связываются студент и олимпиада, а после проверки работ сохраняются баллы и итоговая категория.

Обязательные технические условия первого этапа: FastAPI, SQLAlchemy, клиент-серверная PostgreSQL, JWT, версии v1/v2 и отдельная автоматическая документация. Для основных операций предусмотрены HTTP-методы GET, POST, PUT/PATCH и DELETE. Критерии приёмки включают выполнение сценария от заявки до публикации, создание настоящих DOCX/XLSX и проверку ограничений доступа [23].

1.1 Уточнения и критерии приёмки

В исходном перечне категорий отдельно не указан студент, хотя далее описаны действия зарегистрированного участника и создание студенческих учётных записей. Поэтому для участника принята роль student. Самостоятельная публичная регистрация не добавлена: студента создаёт преподаватель или администратор, преподавателя — администратор.

Задание не содержит общих порогов для назначения победителя и призёра. Категория результата вводится ответственным преподавателем по положению конкретной олимпиады. Система проверяет допустимое значение категории и корректность баллов, но не заменяет решение организатора произвольной формулой.

Приняты три состояния протокола: draft — формируется, prepared — подготовлен, published — опубликован. В prepared результаты и заявки заблокированы. Исправление возможно после отзыва в draft. Опубликованный протокол не отзывается, поскольку в ТЗ отзыв разрешён только для подготовленных протоколов.

Критерии приёмки: корректный цикл от заявки до публикации, разграничение ответственности, скрытие результатов до публикации, читаемые Word/Excel, не менее 50 записей на основную сущность и покрытие серверного кода не ниже 80%. Реестр требований и матрица находятся в docs/TEST_PLAN.md; действия и результаты — в docs/TEST_CASES.md и docs/TEST_REPORT.md.

2 АНАЛИТИЧЕСКАЯ ЧАСТЬ

2.1 Цели, функции и задачи системы

Главная задача системы — хранить согласованный набор сведений о мероприятиях и участии студентов. Данные вводятся один раз и используются при просмотре, публикации и выгрузке отчётов. Повторное ручное составление нескольких несвязанных таблиц заменяется запросами к общей базе.

Функции распределены между модулями авторизации, учётных записей, олимпиад, участия, протоколов и документов. Такое разделение соответствует действиям пользователей: учётная запись определяет исполнителя, карточка описывает мероприятие, заявка связывает его со студентом, протокол содержит итог, а вложения хранят положение и бланки.

Работа с API на первом этапе выполняется через Swagger или обычный HTTP-клиент. Отдельная клиентская программа не включена. Будущее веб-приложение сможет обращаться к тем же маршрутам, не изменяя данные прямыми запросами к таблицам.

Задачи обработки: отбор по году, поиск и пагинация v2, уникальность заявки, полнота протокола, права и выгрузки. Используются небольшие функции и несколько файлов: каждая модель находится в отдельном файле models/, каждая схема — в отдельном файле schemas/, обработчики — в main.py. Дополнительные уровни репозиториев и фоновые процессы для учебного объёма не вводятся.

2.2 Описание предметной области

Мероприятие имеет одного ответственного преподавателя и может иметь много участников. Студент может участвовать в нескольких мероприятиях. Такая связь «многие ко многим» раскрыта через таблицу participations. В ней находятся баллы и результат конкретного студента на конкретной олимпиаде.

Протокол не выделен в отдельную таблицу: у мероприятия существует один текущий набор результатов и один статус. Это уменьшает число сущностей без потери функций первого этапа. История версий протокола и повторное опубликование исправлений не входят в выданное ТЗ.

Документ относится к одной олимпиаде и имеет вид: положение, задание, сертификат или грамота. Количество вложений не ограничивается одним бланком. Сами файлы находятся в storage, метаданные — в attachments. При замене сохраняется ID документа, содержимое записывается под новым случайным именем, старый файл удаляется после успешного сохранения.

Исходные данные определены персональным ТЗ и методическими материалами [23, 24, 25]. Реальные результаты и опросы сотрудников не предоставлялись. В пустой базе инициализация создаёт 50 пользователей, 50 олимпиад, 50 участий, 50 документов. Пользователи: один администратор, два преподавателя и 47 студентов. Все сведения вымышлены. Реляционные связи отделяют личные данные от результатов участия [1, 3, 4, 9, 20].

2.3 Пользователи и разграничение доступа

Гость не имеет учётной записи и получает только общедоступные сведения. Студент работает со своим профилем и своими заявками. Преподаватель создаёт студенческие аккаунты и отвечает за мероприятия, где его идентификатор указан в teacher_id. Администратор обслуживает всю систему и выполняет публикацию.

Аутентификация подтверждает личность по логину и паролю, затем по JWT. Авторизация отдельно проверяет роль и принадлежность объекта. Недостаточно проверить только роль teacher: иначе любой преподаватель получил бы возможность менять результаты чужой олимпиады. Поэтому общая функция check_owner сравнивает teacher_id и id пользователя.

Профиль изменяется отдельной схемой без полей role, is_active и token_version. Лишние поля отклоняются. В публичном протоколе выдаются ФИО, баллы и результат; email, дата рождения и хеш отсутствуют. Проверки выполняются на сервере независимо от HTTP-клиента [10, 19].

На рисунке 1 показаны варианты использования для четырёх актёров. Стрелки зависимостей между прецедентами не введены, поскольку основные операции представлены самостоятельными вариантами использования. Доступ к общим публичным операциям сохраняется и у зарегистрированных пользователей.

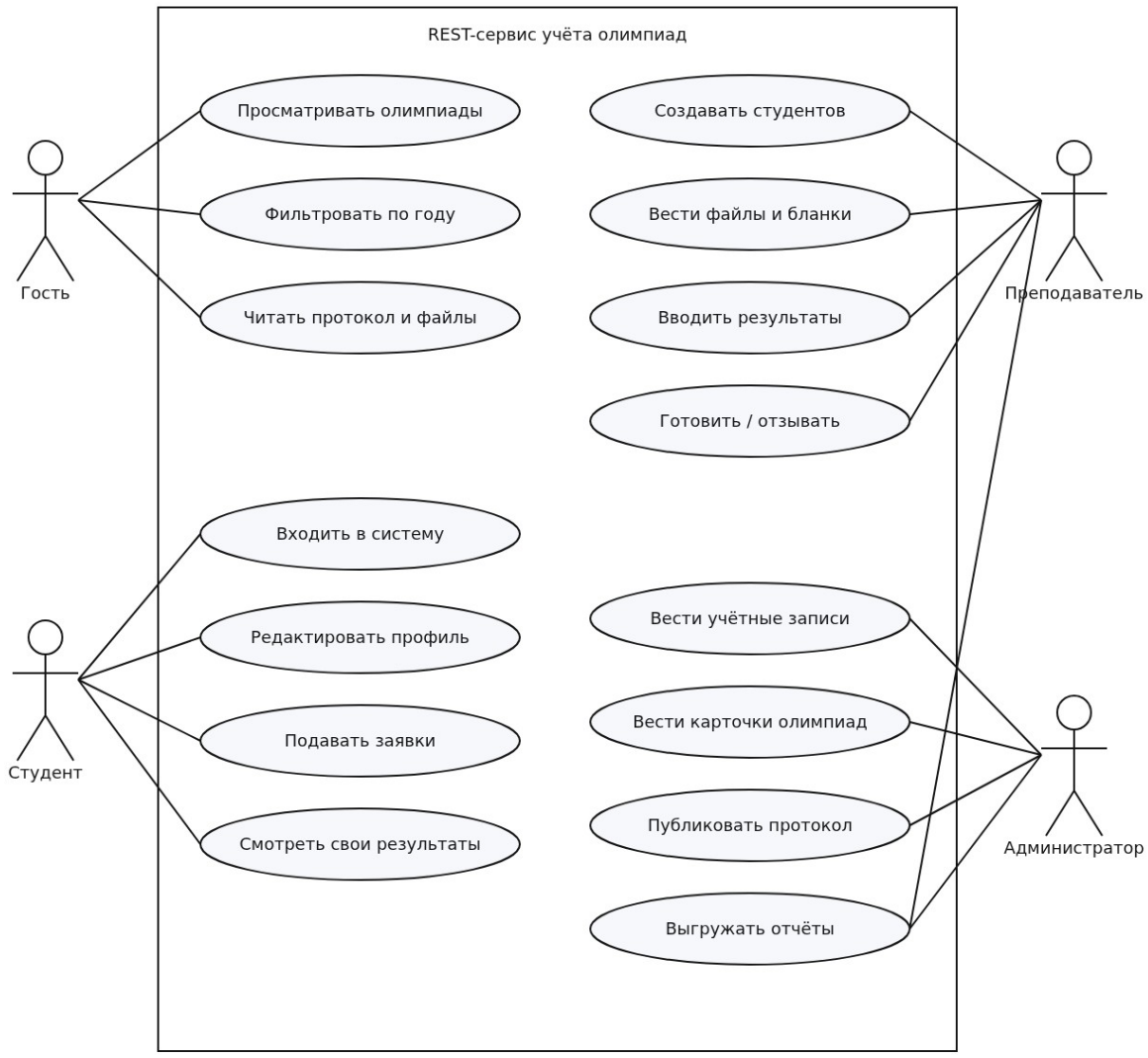


Рисунок 1 — Варианты использования REST-сервиса

2.4 Внутренняя архитектура

Структура показана на рисунке 2. FastAPI получает запрос и проверяет тело через схемы Pydantic из schemas/. Обработчик в main.py получает сессию SQLAlchemy и текущего пользователя. Пароли и JWT вынесены в auth.py, модели — в models/, подключение — в database.py, Word/Excel — в reports.py.

Модели SQLAlchemy описывают четыре таблицы. Они не передаются клиенту напрямую без выбора полей: для основных объектов применяются выходные схемы, для протоколов и заявок явно составляются словари. Это позволяет отделить физическую структуру таблицы users от открытого ответа API.

Сессия БД создаётся на запрос и закрывается после его завершения. При нарушении уникальности выполняется rollback и возвращается 409. Изменения статуса, заявки и результата используют блокировку строки олимпиады. Все версии работают с одной базой; номер API не обозначает отдельную копию данных.

Модульный подход остаётся простым: маршруты — обычные функции, классы используются для ORM и схем данных. Повторяющиеся проверки владельца, статуса и сохранения находятся рядом с маршрутами в main.py. Такое разделение соответствует назначению частей программы [8, 11, 16, 21]. Используется одна PostgreSQL, распределённое хранение в данном проекте не требуется [2, 9].

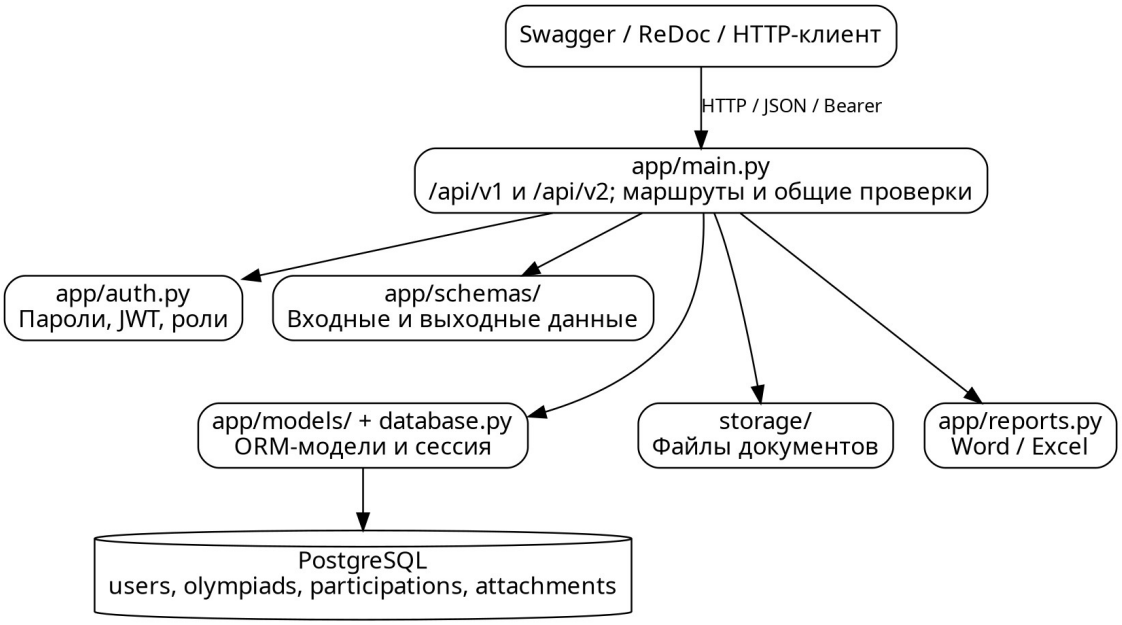


Рисунок 2 — Структура приложения и движение данных

2.5 Проектные решения по тестированию

Проверки должны обнаруживать ошибки в действиях и правах, а не только подтверждать запуск процесса. Для этого выбран pytest, FastAPI TestClient и отдельная PostgreSQL-база. Тесты создают минимальный набор аккаунтов и событий, выполняют HTTP-запросы и проверяют коды, структуру ответа и сохранённое состояние [5, 7, 13, 14, 15, 17].

Положительные случаи дополнены неверными паролями/JWT, отсутствующими записями, повторной заявкой, неверными баллами и датами, чужим преподавателем и публикацией неподготовленного протокола. Для файлов проверены содержимое, расширение, размер, замена и удаление; Word/Excel повторно открываются библиотеками. Подход к ошибочным и граничным значениям соответствует задачам систематического тестирования [5, 7, 13, 14, 15, 17].

Каждый тест использует внешнюю транзакцию и savepoint. Вызов commit внутри маршрута не делает тестовые данные постоянными: после теста внешняя транзакция откатывается. Каталог загрузок заменяется временным. Тестовая БД имеет имя с суффиксом _test, что дополнительно препятствует случайному запуску на рабочей базе.

Критерий завершения — успешный набор тестов, отсутствие известных блокирующих дефектов и покрытие не ниже 80%. Порог задан в pytest.ini и автоматически проверяется при запуске. План, матрица и сценарии приведены ниже в этом разделе и отдельно в TEST_PLAN.md и TEST_CASES.md; фактический отчёт — в TEST_REPORT.md. Для обязательных дополнений выполнен цикл падающих тестов, реализации и повторного прогона; TDD для всей предыдущей истории не заявляется.

Ниже приведены обязательные документы тестирования по п. 2.7.5 методических рекомендаций: реестр требований, Test Plan, матрица трассировки и сценарии. Отдельные файлы TEST_PLAN.md и TEST_CASES.md сохранены для самостоятельного использования. Идентификаторы в обоих представлениях совпадают.

Перечень функций приведён в таблице 2, план — в таблице 3, матрица — в таблице 4, набор данных — в таблице 5. Для каждого сценария далее указаны предусловия, действия, ожидаемый и фактический результат.

Таблица 2 — Функции первого этапа, подлежащие проверке

ID требования	**Описание функции**
R01	Список, карточка, фильтр олимпиад по году
R02	Общедоступный протокол только после публикации
R03	Положения и задания прошлых лет
R04	Вход, профиль, редактирование своего профиля
R05	Участие в нескольких олимпиадах, отсутствие дубликатов
R06	Собственные опубликованные результаты
R07	Создание преподавателей и редактирование аккаунтов администратором
R08	Карточки: название, ответственный, даты, CRUD
R09	Просмотр статусов и публикация только подготовленного протокола
R10	Протокол Word и Excel
R11	Годовой отчёт с результатами
R12	Прикрепление созданного преподавателем положения
R13	Создание студентов со всеми личными и учебными полями
R14	Баллы, категории результата, подготовка и отзыв протокола
R15	Несколько бланков грамот/сертификатов и их изменение
R16	Версии API, Swagger/ReDoc, JWT, HTTP-методы
R17	PostgreSQL и ORM, без встроенной БД
R18	Тесты, изоляция, минимум 80% покрытия
R19	Не менее 50 записей для каждой основной сущности

Таблица 3 — Test Plan первого этапа API

ID	Объект / требования	Стратегия	Порядок проведения	Критерии начала	Критерии окончания
TC01	Хеши, вход и JWT R04 R16	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC01; падений нет
TC02	Профиль и управление аккаунтами R04 R07	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC02; падений нет
TC03	Создание пользователей R07 R13	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC03; падений нет
TC04	Карточки и полный PUT R08 R16	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC04; падений нет
TC05	Список и версии R01 R16	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC05; падений нет

Продолжение таблицы 3

ID	Объект / требования	Стратегия	Порядок проведения	Критерии начала	Критерии окончания
TC06	Заявки R05 R06	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC06; падений нет
TC07	Ответственность и чужие данные R05 R09 R14	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC07; падений нет
TC08	Цикл протокола R02 R06 R09 R14	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC08; падений нет
TC09	Word/Excel и годовой отчёт R10 R11	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC09; падений нет
TC10	Положения и бланки R03 R12 R15	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки TC10; падений нет

Продолжение таблицы 3

ID	Объект / требования	Стратегия	Порядок проведения	Критерии начала	Критерии окончания
ТС11	Задания прошлых лет R03	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки ТС11; падений нет
ТС12	Валидация и хранение файлов R03 R15	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки ТС12; падений нет
ТС13	Документация и доступ v2 R16 R17 R18	API; позитивная и негативная; интеграционная	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки ТС13; падений нет
ТС14	Объём данных и повторный запуск R17 R18 R19	Интеграционная; проверка данных	Фикстуры → запросы → сверка → откат	Доступна тестовая БД; готовы фикстуры	Выполнены ожидаемые проверки ТС14; падений нет

Порядок проведения: сначала хеши, JWT и роли, затем CRUD и версии, полный путь протокола, файлы и отчёты, после — регрессия и покрытие. Условия завершения: ни одного падения выбранного набора и не менее 80% покрытия серверных маршрутов и проверок. Нагрузочные испытания не проводились. Время и статистика повторного прогона приведены в разделе 3.6.

Таблица 4 — Матрица трассировки требований и тестовых сценариев

Req. ID	**TC01**	**TC02**	**TC03**	**TC04**	**TC05**	**TC06**	**TC07**
R01	—	—	—	—	+	—	—
R02	—	—	—	—	—	—	—
R03	—	—	—	—	—	—	—
R04	+	+	—	—	—	—	—
R05	—	—	—	—	—	+	+
R06	—	—	—	—	—	+	—
R07	—	+	+	—	—	—	—
R08	—	—	—	+	—	—	—
R09	—	—	—	—	—	—	+
R10	—	—	—	—	—	—	—
R11	—	—	—	—	—	—	—
R12	—	—	—	—	—	—	—
R13	—	—	+	—	—	—	—
R14	—	—	—	—	—	—	+
R15	—	—	—	—	—	—	—
R16	+	—	—	+	+	—	—
R17	—	—	—	—	—	—	—
R18	—	—	—	—	—	—	—
R19	—	—	—	—	—	—	—

Продолжение таблицы 4

Req. ID	**TC08**	**TC09**	**TC10**	**TC11**	**TC12**	**TC13**	**TC14**
R01	—	—	—	—	—	—	—
R02	+	—	—	—	—	—	—
R03	—	—	+	+	+	—	—
R04	—	—	—	—	—	—	—
R05	—	—	—	—	—	—	—
R06	+	—	—	—	—	—	—
R07	—	—	—	—	—	—	—
R08	—	—	—	—	—	—	—
R09	+	—	—	—	—	—	—
R10	—	+	—	—	—	—	—
R11	—	+	—	—	—	—	—
R12	—	—	+	—	—	—	—
R13	—	—	—	—	—	—	—
R14	+	—	—	—	—	—	—
R15	—	—	+	—	+	—	—
R16	—	—	—	—	—	+	—
R17	—	—	—	—	—	+	+
R18	—	—	—	—	—	+	+
R19	—	—	—	—	—	—	+

Знак «+» обозначает проверку требования соответствующим сценарием; «—» — отсутствие прямой связи. Сценарии не равны отдельным тестам: параметризация разворачивает их в 72 проверки.

Таблица 5 — Входные данные и ожидаемая обработка

Объект	**Корректные данные**	**Неверные / граничные данные**	**Ожидаемая обработка**
Студент	student; TestPass2026!; student@example.org; курс 3	Логин missing; пароль wrong; курс 0/7; email без @	Вход 200 или 401; валидация 422
Дата и ответственный	Начало и конец сегодня; активный teacher	Конец до начала; student вместо teacher; NULL	422; существующая запись не изменена
Баллы	95.5; winner	−1; 1000000; 1.111; unknown	422; корректные баллы сохраняются
Список v2	page=1; page_size=10; year=2026	page=0; page_size=0/101; year=1999	422; допустимые значения дают нужный отбор
Файлы	Настоящие DOCX/XLSX; UTF-8 TXT	Пустой файл; .exe; ложная сигнатура; 5 МБ + 1	201 для корректного; 422 или 413 для неверного
Путь и права	rules.txt; ответственный teacher	../../escape.txt; чужой преподаватель	Безопасное имя; чужое изменение — 403

Фикстуры создают пять аккаунтов и событие на текущую дату в откатываемой транзакции. Для проверки объёма отдельно вызывается seed_demo в пустой тестовой транзакции: по 50 записей и 50 реальных Word-файлов. Office-файлы создаются библиотеками, ошибочные содержат заведомо неверные байты. Все данные вымышлены.

TC01 — Хеши, вход и JWT

Требования: R04 R16. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для TC14 — пустая тестовая транзакция.

Шаги: Проверить разные соли; верный/неверный пароль; повреждённый, просроченный токен и отсутствующий аккаунт.

Ожидаемый результат: Верный вход 200; неверный 401; исходный пароль не восстанавливается.

Автоматизация: test_password_hashes; test_login_profile; test_bad_login; test_inactive_login; test_invalid_token; test_missing_user_and_expired_token. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС02 — Профиль и управление аккаунтами

Требования: R04 R07. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Изменить ФИО/email; передать role/null; администратором сбросить пароль/заблокировать аккаунт; попытаться заблокировать себя.

Ожидаемый результат: Свои поля 200; лишние/null 422; старый JWT 401; самоблокировка 409.

Автоматизация: test_profile; test_admin_accounts; test_duplicate_profile_email. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС03 — Создание пользователей

Требования: R07 R13. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Создать студента от трёх ролей; создать преподавателя; повторить логин; передать некорректные данные студента.

Ожидаемый результат: 201 для персонала, 403 для студента; преподавателя создаёт только admin; дубль 409; неверные поля 422.

Автоматизация: test_create_student; test_create_teacher_and_duplicate; test_user_validation. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС04 — Карточки и полный PUT

Требования: R08 R16. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Создать, прочитать, PATCH, PUT, удалить пустую олимпиаду; повторить от чужой роли; передать неверные даты и ответственного.

Ожидаемый результат: 201/200/204, после удаления 404; чужие роли 403; неверные поля 422; prepared не меняется (409).

Автоматизация: test_crud; test_crud_forbidden; test_invalid_olympiad; test_cannot_modify_finalized; test_inactive_teacher; test_replace_olympiad. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС05 — Список и версии

Требования: R01 R16. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Добавить событие прошлого года; запросить год, поиск, страницу; передать page=0, page_size=0/101, year=1999.

Ожидаемый результат: Точный отбор; v1 массив, v2 items/total/page/page_size; неверные границы 422.

Автоматизация: test_filter_and_versions; test_pagination_validation. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС06 — Заявки

Требования: R05 R06. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Подать заявку, повторить, отменить; подать после завершения; попытаться записать преподавателя.

Ожидаемый результат: 201; дубль/завершённый приём 409; своя отмена 204; преподаватель 403.

Автоматизация: test_enrollment_and_deletion; test_complete_workflow. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС07 — Ответственность и чужие данные

Требования: R05 R09 R14. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Чужим преподавателем читать/править заявки, протокол и баллы; другим студентом удалить чужую заявку.

Ожидаемый результат: 403; список преподавателя содержит только свои мероприятия.

Автоматизация: test_ownership. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС08 — Цикл протокола

Требования: R02 R06 R09 R14. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Записаться; попытаться подготовить без результата; заполнить; подготовить/отозвать; подготовить и опубликовать администратором.

Ожидаемый результат: До публикации 404/null; неверные переходы 409; публикация преподавателем 403; после публикации баллы открыты.

Автоматизация: test_complete_workflow; test_protocol_incomplete_future_and_validation; test_not_found. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС09 — Word/Excel и годовой отчёт

Требования: R10 R11. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Скачать оба формата, открыть библиотеками, проверить 95.5 и ФИО; пустой год, неизвестный формат; ФИО с формулой.

Ожидаемый результат: Читаемые DOCX/XLSX; пустой отчёт допустим; формат pdf 422; формула остаётся строкой; годовой отчёт преподавателю 403.

Автоматизация: test_complete_workflow; test_excel_formula_safety; test_draft_export. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС10 — Положения и бланки

Требования: R03 R12 R15. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Загрузить готовый документ; скачать; заменить грамоту по ID; добавить две грамоты; удалить; повторить чужим преподавателем.

Ожидаемый результат: 201/200/204; номер при замене сохранён, старый файл удалён; чужой доступ 403.

Автоматизация: test_upload_download_delete; test_edit_award_blank; test_multiple_awards_and_permissions. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС11 — Задания прошлых лет

Требования: R03. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Загрузить task текущего года, получить список/файл; установить прошлый год и повторить.

Ожидаемый результат: Текущее задание скрыто (404, пустой список), прошлогоднее доступно (200).

Автоматизация: test_past_tasks. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС12 — Валидация и хранение файлов

Требования: R03 R15. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Передать пустой/поддельный/большой файл; неверный тип; корректные Office-файлы; ../../ в имени; отсутствующий физический файл.

Ожидаемый результат: Неверные файлы 422, размер 413, корректные 201, путь остаётся в хранилище, отсутствие 404.

Автоматизация: test_file_validation; test_office_uploads; test_missing_file; test_no_path_traversal. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС13 — Документация и доступ v2

Требования: R16 R17 R18. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: Получить OpenAPI обеих версий; гостем создать олимпиаду; выполнить запись через v2 и проверить роль.

Ожидаемый результат: Схемы 200, гость 401; CRUD v2 работает, роль студента не открывает список аккаунтов.

Автоматизация: test_versioned_documentation; test_guest_cannot_create_olympiad; test_v2_security_and_write. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

ТС14 — Объём данных и повторный запуск

Требования: R17 R18 R19. Предусловия: отдельная PostgreSQL-транзакция и фикстуры accounts, headers, olympiad; для ТС14 — пустая тестовая транзакция.

Шаги: В пустой тестовой БД выполнить seed_demo; посчитать каждую таблицу, открыть все файлы; повторить.

Ожидаемый результат: По 50 записей; четыре типа документов; повтор не меняет количества.

Автоматизация: test_demo_has_required_records. Фактический результат повторного прогона: соответствует ожидаемому, успешно.

3 ПРОЕКТНАЯ ЧАСТЬ

3.1 Стек технологий

Проверенная среда — Python 3.12.14 и PostgreSQL 17.11. Для обычной установки указаны Python 3.11+ и PostgreSQL 15+. FastAPI 0.115.12 создаёт маршруты и OpenAPI; Pydantic 2.11.7 проверяет данные; SQLAlchemy 2.0.41 описывает ORM; psycopg 3.2.9 подключается к клиент-серверной СУБД [1, 6, 9, 18, 20].

PyJWT 2.10.1 применяется для подписанных токенов HS256. PBKDF2 реализован средствами стандартной библиотеки hashlib: дополнительный пакет для хранения паролей не нужен. python-multipart разбирает формы авторизации и загрузки. python-dotenv читает локальные настройки, не помещая их в исходный код.

Word формируется библиотекой python-docx 1.2.0, Excel — openpyxl 3.1.5. Тесты используют pytest 8.4.1, pytest-cov 6.2.1 и httpx 0.28.1. Рабочие зависимости перечислены в requirements.txt, тестовые — в requirements-dev.txt. Requests и отдельный консольный клиент не требуются.

Используется синхронная SQLAlchemy-сессия и обычные функции def. Для небольшого учебного сервиса это проще для чтения и отладки. PostgreSQL 17 устанавливается как обычная служба операционной системы. Python-приложение работает в venv и подключается к ней через psycopg. Для запуска требуются Python и PostgreSQL. Используются обычные функции Python и явные проверки [12].

3.2 Модель и ограничения базы данных

Модель содержит users, olympiads, participations и attachments. На рисунке 3 показаны ключи и связи. Словарь полей приведён в таблицах 6–9. Структура сверена с моделями и настоящей PostgreSQL проверочного запуска. Создание таблиц выполняется Base.metadata.create_all в init_db.py.

Учётная запись хранит общие сведения и роль. Для студента обязательны учебные поля; для преподавателя они могут быть пустыми. Обязательность зависит от роли и проверяется схемой создания. Логин и email уникальны. Поле created_by описывает создателя записи, token_version используется для отзыва токенов.

Олимпиада ссылается на ответственного преподавателя. В заявке есть уникальная пара olympiad_id/student_id. Именно ограничение БД препятствует дублированию при одновременных запросах. Внешние ключи не допускают ссылок на отсутствующие записи. Удаление олимпиады с данными блокируется приложением и связями.

При формировании отчёта баллы хранятся как Numeric(8,2), а не как двоичное число с плавающей точкой. В JSON ответ преобразуется к числу для удобства клиента. Категории результата, роли и переходы статусов проверяются приложением; не все правила продублированы SQL CHECK. Это различие отражено в словаре данных.

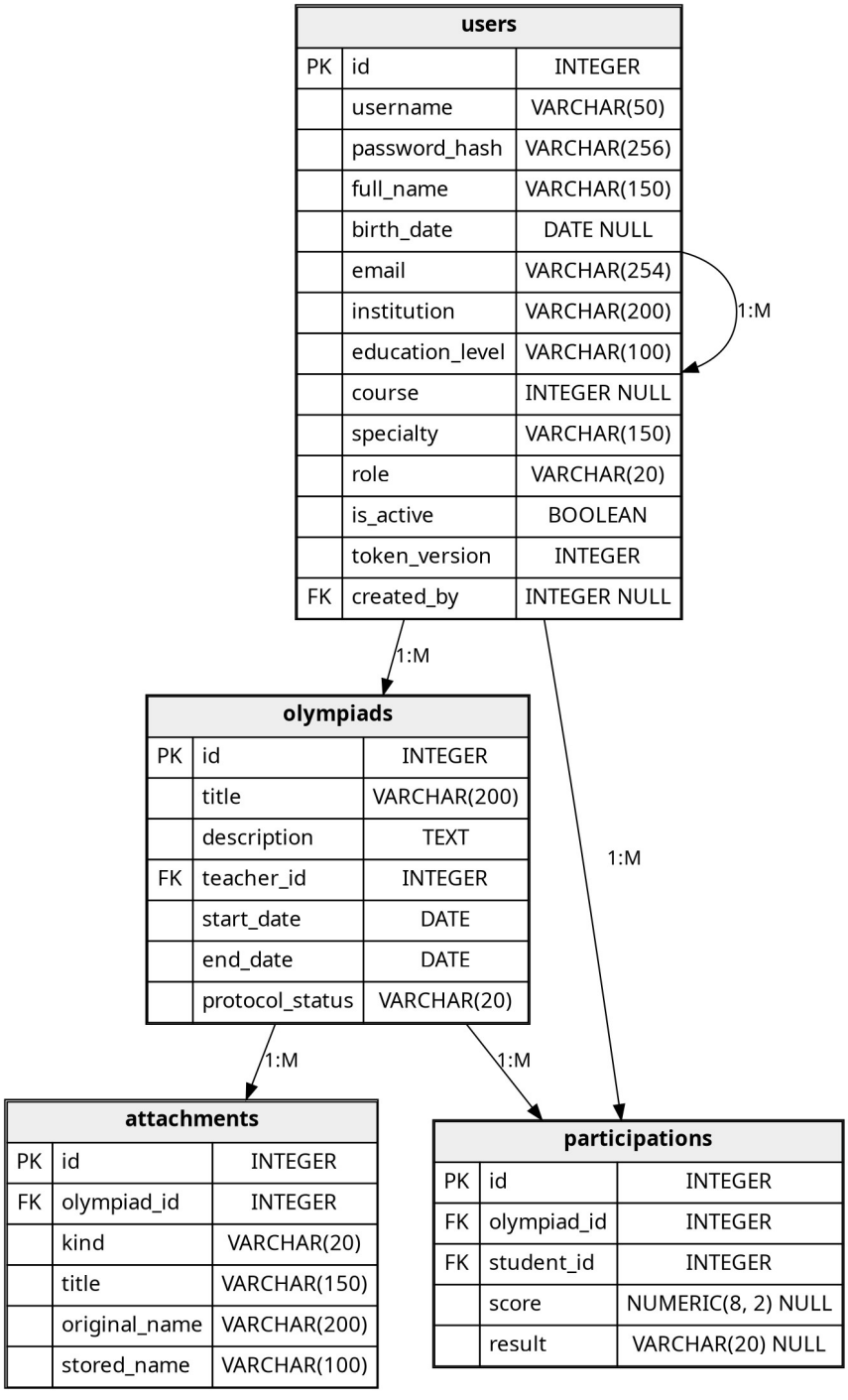


Рисунок 3 — Физическая модель данных PostgreSQL

Состав таблиц и типы столбцов соответствуют app/models/ и PostgreSQL 17. Словарь позволяет сопоставить поля API и физическое хранение.

PK — первичный ключ; FK — внешний ключ; UQ — уникальность. Проверки ролей и большинства входных полей выполняются схемами Pydantic и обработчиками API; ограничения БД перечислены отдельно.

3.2.1 Таблица users

Таблица 6 — Словарь данных: users

Ключ	Имя поля	Тип данных	NULL	Дополнительные условия	Описание
PK	id	INTEGER	нет	БД: последовательность, шаг 1	Идентификатор аккаунта
UQ	username	VARCHAR (50)	нет	БД: UNIQUE	Уникальный логин, 3–50 символов
—	password_hash	VARCHAR (256)	нет	—	PBKDF2-хеш; исходный пароль не хранится
—	full_name	VARCHAR (150)	нет	—	Фамилия, имя, отчество
—	birth_date	DATE	да	API: прошлое; обязательно студенту	Дата рождения; для студента обязательна
UQ	email	VARCHAR (254)	нет	БД: UNIQUE	Уникальная электронная почта
—	institution	VARCHAR (200)	нет	Приложение: пустая строка по умолчанию	Учебное заведение

Продолжение таблицы 6

Ключ	Имя поля	Тип данных	NULL	Дополнительные условия	Описание
—	education_level	VARCHAR (100)	нет	Приложение: пустая строка по умолчанию	Уровень образования
—	course	INTEGER	да	API: 1–6	Курс обучения, 1–6
—	specialty	VARCHAR (150)	нет	Приложение: пустая строка по умолчанию	Код и название специальности
—	role	VARCHAR (20)	нет	API: admin / teacher / student	admin / teacher / student
—	is_active	BOOLEAN	нет	Приложение: True	Признак действующего аккаунта
—	token_version	INTEGER	нет	Приложение: 0; увеличивается при отзыве	Версия токенов для отзыва сеансов
FK → users.id	created_by	INTEGER	да	—	Кто создал аккаунт; допускается NULL для начального администратора

3.2.2 Таблица olympiads

Таблица 7 — Словарь данных: olympiads

Ключ	Имя поля	Тип данных	NULL	Дополнительные условия	Описание
PK	id	INTEGER	нет	БД: последовательность, шаг 1	Идентификатор мероприятия
—	title	VARCHAR (200)	нет	—	Название
—	description	TEXT	нет	Приложение: пустая строка по умолчанию	Описание

Продолжение таблицы 7

Ключ	Имя поля	Тип данных	NULL	Дополнительные условия	Описание
FK → users.id	teacher_id	INTEGER	нет	—	Ответственный действующий преподаватель
—	start_date	DATE	нет	—	Дата начала и источник года для фильтра
—	end_date	DATE	нет	БД: не раньше start_date	Дата окончания, не раньше начала
—	protocol_status	VARCHAR (20)	нет	Приложение: draft по умолчанию	draft / prepared / published

3.2.3 Таблица participations

Таблица 8 — Словарь данных: participations

Ключ	Имя поля	Тип данных	NULL	Дополнительные условия	Описание
PK	id	INTEGER	нет	БД: последовательность, шаг 1	Идентификатор заявки
FK → olympiads.id	olympiad_id	INTEGER	нет	БД: UNIQUE пары olympiad_id/student_id	Ссылка на мероприятие
FK → users.id	student_id	INTEGER	нет	БД: UNIQUE пары olympiad_id/student_id	Ссылка на студента
—	score	NUMERIC (8, 2)	да	БД: ≥0; API: ≤999999.99, 2 знака	Баллы 0… 999999.99, два десятичных знака; NULL до ввода
—	result	VARCHAR (20)	да	API: winner / prize / participant	winner / prize / participant; NULL до ввода

3.2.4 Таблица attachments

Таблица 9 — Словарь данных: attachments

Ключ	**Имя поля**	**Тип данных**	**NULL**	**Дополнительные условия**	**Описание**
PK	id	INTEGER	нет	БД: последовательность, шаг 1	Идентификатор документа
FK → olympiads.id	olympiad_id	INTEGER	нет	—	Владелец документа — мероприятие
—	kind	VARCHAR (20)	нет	API: четыре типа документов	regulation / task / certificate / award

Продолжение таблицы 9

Ключ	Имя поля	Тип данных	NULL	Дополнительные условия	Описание
—	title	VARCHAR (150)	нет	—	Отображаемое название
—	original_name	VARCHAR (200)	нет	—	Исходное безопасное имя для скачивания
—	stored_name	VARCHAR (100)	нет	—	Случайное имя физического файла, не выдаётся API

3.2.5 Связи и ограничения

users → olympiads: один преподаватель отвечает за много мероприятий. users → participations: один студент имеет много заявок. olympiads → participations и olympiads → attachments: связи один-ко-многим. users.created_by ссылается на users.id.

В БД действуют уникальности username, email и пары (olympiad_id, student_id), CHECK end_date ≥ start_date, CHECK score ≥ 0, NOT NULL и внешние ключи. Значения ролей, категории результата, верхняя граница баллов, активность ответственного, обязательность учебных полей и переходы статусов проверяются приложением. Ошибки целостности откатываются и возвращают 409.

Удаление олимпиад с зависимостями блокируется. Файловое содержимое хранится вне БД: attachments содержит только метаданные. Схема соответствует третьей нормальной форме в принятом объёме: данные студента не дублируются в каждой заявке; название мероприятия не дублируется в результатах; справочные роли и типы имеют небольшой фиксированный набор и не вынесены в отдельные таблицы.

3.3 Файловая структура и назначение модулей

Корень содержит README.md, requirements.txt, requirements-dev.txt, pytest.ini, database.sql, init_db.py и пример .env. app содержит сервер, tests — автоматические проверки, docs — необходимые документы сдачи, storage — загруженные файлы. Генераторы пояснительной записки и снимков в репозиторий не включены.

В main.py находятся обработчики входа, профиля, пользователей, олимпиад, участия, результатов, протоколов и документов. models/ содержит четыре ORM-класса; schemas/ — входные и выходные модели; auth.py — пароль и JWT; database.py — сессию и настройки; reports.py — табличные выгрузки.

Корневое приложение монтирует api_v1 и api_v2. Каждый получает общие маршруты через include_router, а список олимпиад регистрируется отдельно для нужного контракта. Документация создаётся на лету: /api/v1/openapi.json и /api/v2/openapi.json. Статические копии этих схем в комплект не добавлены.

Тесты сгруппированы по учётным записям, олимпиадам, протоколам, вложениям, контракту и обязательным требованиям. conftest.py создаёт отдельное соединение PostgreSQL, транзакции и фикстуры; зависимости переопределяются у обоих подприложений. Это позволяет запускать тесты без отдельного HTTP-процесса.

Файловая структура обязательной части:

app/: main.py, auth.py, database.py, reports.py; models/ и schemas/ — отдельный файл для каждого класса;

tests/: conftest.py, проверки аккаунтов, олимпиад, протоколов, вложений, контракта и обязательных требований;

docs/: записка DOCX/PDF, API.md, INSTALL.md, TEST_PLAN.md, TEST_CASES.md, TEST_REPORT.md и используемые снимки;

корень: README.md, requirements.txt, requirements-dev.txt, pytest.ini, database.sql, init_db.py, .env.example; storage/ — пользовательские файлы.

Назначение каждого класса и его файл указаны в таблице 10; фикстуры и очистка их состояния — в таблице 11.

Таблица 10 — Классы приложения и их файлы

Класс	**Отдельный файл**	**Назначение**
Base	app/database.py	Основа декларативных ORM-моделей
Attachment	app/models/attachment.py	Таблица attachments
Olympiad	app/models/olympiad.py	Таблица olympiads
Participation	app/models/participation.py	Таблица participations
User	app/models/user.py	Таблица users
AccountUpdate	app/schemas/account_update.py	Входная схема
ApiModel	app/schemas/api_model.py	Базовые настройки валидации
AttachmentRead	app/schemas/attachment_read.py	Выходная схема ответа

Продолжение таблицы 10

Класс	**Отдельный файл**	**Назначение**
OlympiadCreate	app/schemas/olympiad_create.py	Входная схема
OlympiadPage	app/schemas/olympiad_page.py	Выходная схема ответа
OlympiadRead	app/schemas/olympiad_read.py	Выходная схема ответа
OlympiadUpdate	app/schemas/olympiad_update.py	Входная схема
ProfileUpdate	app/schemas/profile_update.py	Входная схема
ProtocolUpdate	app/schemas/protocol_update.py	Входная схема
ResultUpdate	app/schemas/result_update.py	Входная схема
UserCreate	app/schemas/user_create.py	Входная схема
UserRead	app/schemas/user_read.py	Выходная схема ответа

Таблица 11 — Назначение тестовых фикстур

Фикстура	**Область / назначение**	**Освобождение состояния**
test_engine	Сессия тестов; отдельный PostgreSQL engine, создание таблиц	dispose после набора
db	Каждый тест; внешняя транзакция и Session с savepoint	Откат внешней транзакции
client	Каждый тест; TestClient, подмена get_db у обоих API, временное хранилище	Закрытие клиента; очистка overrides
password_hash	Сессия тестов; один PBKDF2-хеш TestPass2026! для фикстур	Значение только в памяти
accounts	Каждый тест; admin, два teacher, два student	Откат через db
headers	Каждый тест; Bearer-заголовки соответствующих аккаунтов	Значения только в памяти
olympiad	Каждый тест; событие на сегодня с ответственным teacher	Откат через db

3.4 Алгоритм записи на олимпиаду

Схема записи приведена на рисунке 4. Сервер получает Bearer-токен и проверяет существование активного студента. Роль teacher или admin не даёт права записать себя как участника. Мероприятие ищется по идентификатору; отсутствие записи даёт 404.

До добавления заявки блокируется строка олимпиады. Проверяется статус draft и дата окончания. Условие даты позволяет подать заявку в последний день включительно. Затем создаётся Participation с пустыми баллами и результатом. Успех возвращает 201 и идентификатор заявки.

Если такая пара студент–олимпиада уже существует, ограничение UNIQUE вызывает ошибку целостности. Общая функция save откатывает транзакцию и преобразует ошибку в 409. Проверка только предварительным SELECT была бы недостаточна: два одновременных запроса могли бы оба не найти заявку до вставки.

Отмена заявки выполняется отдельным DELETE. Студент может удалить только свою заявку, преподаватель — в своей олимпиаде, администратор — любую при draft. После подготовки протокола состав участников фиксируется. Эти правила проверяются положительными и отрицательными тестами.

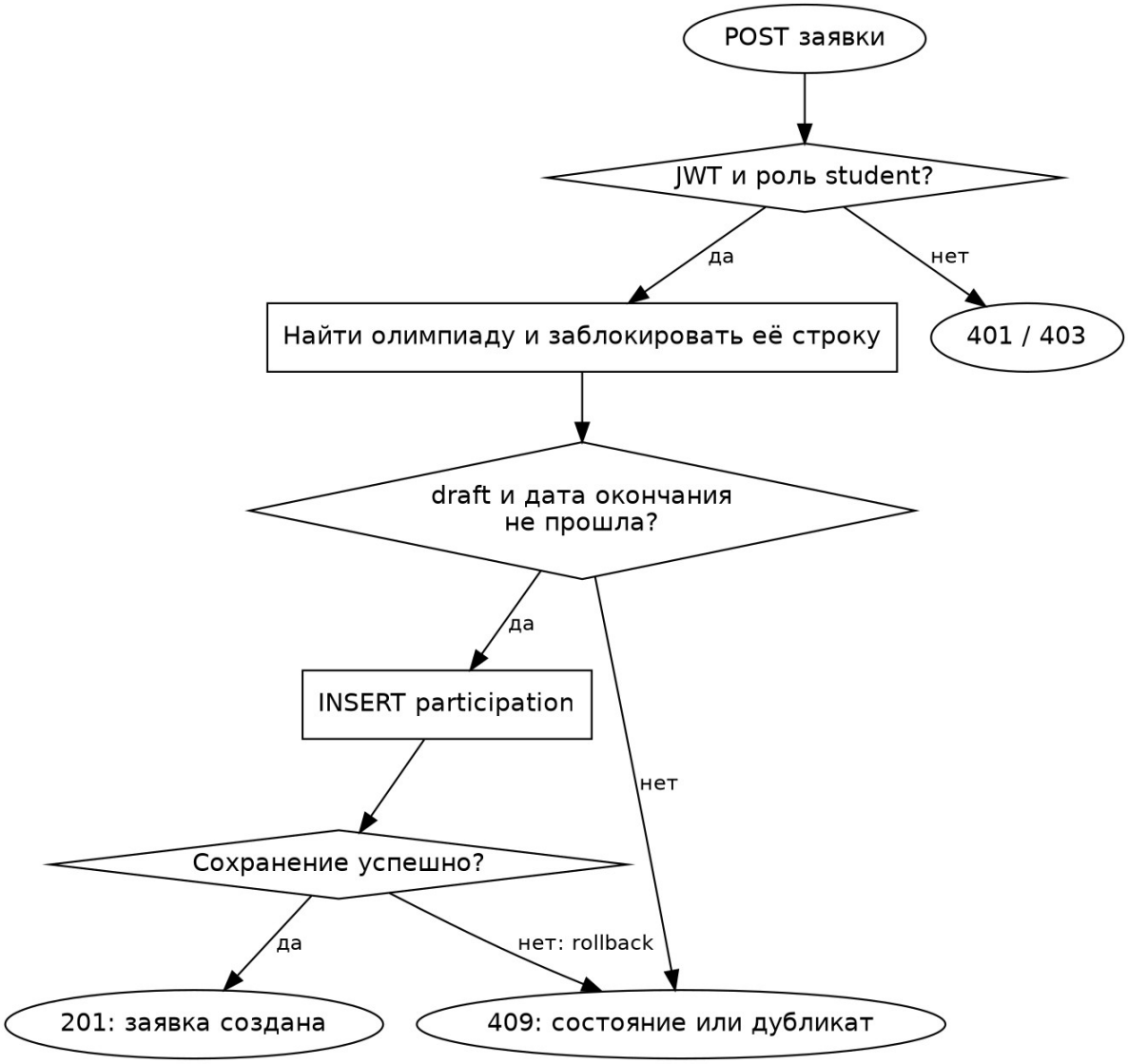


Рисунок 4 — Алгоритм обработки заявки

3.4.1 Подготовка, отзыв и публикация протокола

Модель состояний показана на рисунке 5. Исходное состояние — draft. В нём преподаватель заполняет баллы и категорию результата каждого участника. Список сортируется по убыванию баллов, однако это не назначает призовое место автоматически.

При переходе draft → prepared проверяются ответственность пользователя, наступление даты окончания, наличие участников и заполненность всех результатов. Только после этого сохраняется prepared. Повторный такой же переход и попытка пропустить подготовку приводят к 409.

Ответственный может отозвать prepared в draft. Тогда редактирование становится доступно снова. Переход prepared → published выполняет только администратор. При published публичный маршрут начинает возвращать результаты, а персональный список студента перестаёт скрывать баллы.

Готовность Word/Excel не равнозначна публикации: преподаватель может скачать черновую выгрузку для проверки. Её заголовок содержит текущий статус. Гость получает протокол только через публичный маршрут, который до публикации возвращает 404. Факт появления результата после публикации проверен HTTP-сценарием.

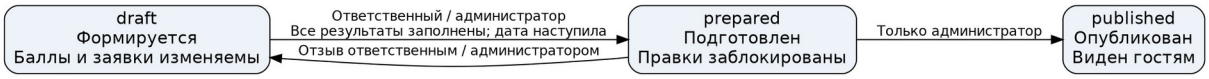


Рисунок 5 — Состояния и переходы протокола

3.4.2 Аутентификация и отзыв токенов

Схема проверки защищённого запроса приведена на рисунке 6. Пароль проверяется через PBKDF2-HMAC-SHA256, соль генерируется отдельно для каждого хеша. Используется 600000 итераций. Результат сравнивается через hmac.compare_digest. Исходный пароль и JWT-ключ в ответы API не включаются.

JWT содержит sub, iat, exp и ver. Принимается только HS256 с настроенным ключом и обязательными полями [6]. Затем из PostgreSQL читается пользователь. Неактивность, отсутствие аккаунта или неверный token_version дают 401.

Сброс пароля или изменение активности администратором увеличивает token_version. Старые токены сразу перестают проходить проверку. Отдельный серверный выход и самостоятельная смена пароля не добавлены: персональное ТЗ API первого этапа их не требует.

Роль и ответственность проверяются после аутентификации. Поэтому корректный JWT чужого преподавателя не открывает доступ к мероприятию. 401 означает необходимость повторного входа, 403 — отсутствие права при известной личности. Срок токена по умолчанию равен 60 минутам.

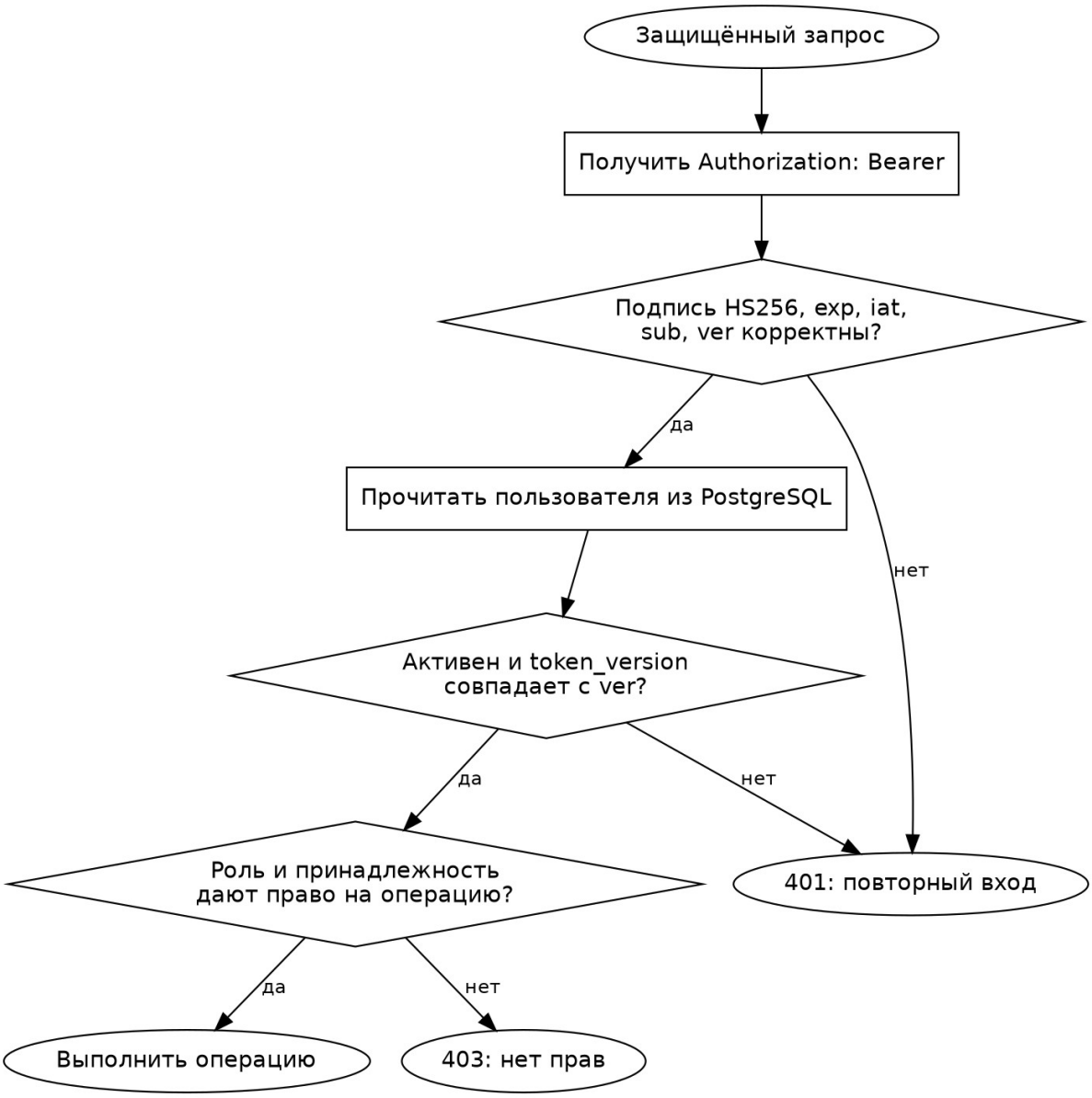


Рисунок 6 — Проверка JWT и прав пользователя

3.5 Документация REST API

Сервис использует JSON и обычные HTTP-методы. Поля запросов и ответов имеют snake_case: teacher_id, start_date, full_name. Токен передаётся в Authorization: Bearer, не в URL. Параметры и схемы доступны в OpenAPI [6].

v1 возвращает массив олимпиад; v2 — объект items, total, page, page_size и поддерживает поиск search. Размер страницы от 1 до 100, номер — от 1. Остальные операции и права общие. v2 расширяет контракт, а не сохраняет ошибки v1.

Swagger: /api/v1/docs и /api/v2/docs. ReDoc: /api/v1/redoc и /api/v2/redoc. Оформление загружается с CDN, поэтому страницам нужен интернет; методы API работают локально. Отдельное руководство docs/API.md содержит операции, роли, поля, коды ошибок и примеры.

В таблице 12 приведён состав операций версии v1. Версия v2 заменяет только GET коллекции олимпиад.

Таблица 12 — Операции REST-сервиса

Метод	**Путь**	**Назначение**
GET	/olympiads	Все олимпиады, фильтр по году
POST	/olympiads	Создать олимпиаду
POST	/auth/token	Получить JWT-токен
GET	/users/me	Посмотреть свой профиль
PATCH	/users/me	Изменить свой профиль
GET	/users	Список учетных записей (администратор)
POST	/users	Создать студента или преподавателя
PATCH	/users/{user_id}	Редактировать учетную запись
GET	/olympiads/{olympiad_id}	Карточка олимпиады
PATCH	/olympiads/{olympiad_id}	Изменить поля карточки
PUT	/olympiads/{olympiad_id}	Заменить данные карточки
DELETE	/olympiads/{olympiad_id}	Удалить пустую олимпиаду

Продолжение таблицы 12

Метод	**Путь**	**Назначение**
POST	/olympiads/{olympiad_id}/ participations	Записаться на олимпиаду
GET	/olympiads/{olympiad_id}/ participations	Заявки для ответственного преподавателя
GET	/users/me/participations	Свои заявки и опубликованные результаты
PUT	/participations/{participation_id}/ result	Записать баллы и результат
DELETE	/participations/{participation_id}	Удалить заявку до подготовки протокола
GET	/protocols	Состояния протоколов (служебный список)
GET	/olympiads/{olympiad_id}/protocol	Опубликованный протокол (общедоступный)
PATCH	/olympiads/{olympiad_id}/protocol	Подготовить, отозвать или опубликовать протокол
GET	/olympiads/{olympiad_id}/protocol-draft	Просмотреть служебный протокол
GET	/olympiads/{olympiad_id}/reports/ protocol	Скачать протокол Word или Excel
GET	/reports/annual	Годовой отчёт по опубликованным результатам
GET	/olympiads/{olympiad_id}/ attachments	Положение и бланки; задания прошлых лет
POST	/olympiads/{olympiad_id}/ attachments	Прикрепить файл
GET	/attachments/{attachment_id}	Скачать общедоступный документ
PUT	/attachments/{attachment_id}	Заменить файл или название
DELETE	/attachments/{attachment_id}	Удалить документ
GET	/olympiads/{olympiad_id}/ attachments/manage	Все файлы ответственного преподавателя

3.5.1 Примеры запросов и обработка ответа

Примеры JSON и HTTP-запросов приведены в docs/API.md. В Swagger можно получить токен кнопкой Authorize и выполнить запрос с проверкой статуса и тела. При программном обращении следует проверять HTTP-код перед разбором JSON; DELETE 204 имеет пустое тело, выгрузка — двоичное.

В карточке передаются title, description, teacher_id, start_date, end_date. Ответственный должен быть действующим преподавателем. При создании студента передаются все личные и учебные поля ТЗ. Профиль и административное редактирование имеют разные схемы; студент не может повысить себе роль.

Файлы передаются multipart/form-data, обычные данные — JSON. Экспорт возвращает двоичный ответ, который сохраняется через response.content. DELETE при успехе возвращает 204 с пустым телом. Попытка вызвать json() для такого ответа является ошибкой клиента, а не требованием сервиса.

Предметные ошибки имеют поле detail. 401 — вход/JWT, 403 — роль/ответственность, 404 — отсутствие/скрытая запись, 409 — конфликт, 413 — размер файла, 422 — данные или формат. Успех: 200, создание: 201, удаление: 204. Схемы и подробные примеры находятся в OpenAPI и docs/API.md.

Вход: POST /api/v1/auth/token, Content-Type: application/x-www-form-urlencoded. Форма: username=teacher1&password=Teacher2026!. Успешный ответ 200: {"access_token":"<JWT>","token_type":"bearer"}. В следующих защищённых запросах нужен Authorization: Bearer <JWT>.

Создание карточки: POST /api/v1/olympiads от администратора. Пример JSON: {"title":"Олимпиада Python","teacher_id":2,"start_date":"2026-12-01","end_date":"2026-12-01"}. Успешный ответ 201 содержит id, title, description, teacher_id, start_date, end_date и protocol_status="draft".

Результат: PUT /api/v1/participations/{id}/result от ответственного преподавателя. JSON: {"score":95.5,"result":"winner"}. Ответ 200 содержит id заявки, olympiad_id, student_id, full_name, score и result. Отрицательный score даёт 422, чужой преподаватель — 403.

Пример пустого отбора v2: GET /api/v2/olympiads?year=2099&page=1&page_size=10. Ответ 200: {"items":[],"total":0,"page":1,"page_size":10}. В v1 аналогичный пустой отбор возвращает [].

До публикации GET /api/v1/olympiads/{id}/protocol от гостя даёт 404 с detail="Протокол ещё не опубликован". Подготовка: PATCH того же пути с {"status":"prepared"}; публикация администратором — с {"status":"published"}.

Протокол Word: GET /api/v1/olympiads/{id}/reports/protocol?format=docx от ответственного или администратора; ответ 200 — настоящий DOCX, а не JSON. Для Excel применяется format=xlsx. Годовой отчёт: GET /api/v1/reports/annual?year=2026&format=xlsx от администратора.

3.5.2 Работа пользователей с API

Гость открывает список, задаёт year, читает карточку и опубликованный протокол. Студент входит как student1, изменяет свой профиль, подаёт заявку и получает свои результаты. До публикации баллы в собственном списке скрыты. Все эти действия доступны через Swagger.

Преподаватель входит как teacher1 или teacher2, создаёт студента через POST /users, прикрепляет готовое положение, задания и бланки, вводит score/result, готовит и отзывает prepared-протокол своей олимпиады. Бланк создаётся в текстовом редакторе и загружается; PUT /attachments/{id} позволяет заменить его без изменения номера.

Администратор входит как admin, создаёт преподавателей, правит аккаунты и карточки, получает список /protocols, публикует подготовленные результаты и выгружает годовой отчёт. Демонстрационные пароли: Admin2026!, Teacher2026!, Student2026! соответственно. Студенты student1… student47; все данные учебные.

Рабочий путь: создать/выбрать мероприятие на сегодня → подать заявку студентом → указать баллы преподавателем → prepared → published администратором → получить публичный протокол. Читаемые Word/Excel проверяются отдельными запросами. Снимки API и фактического запроса приведены в приложении 2.

3.5.3 Руководство по развёртыванию

Запуск не использует Docker. Требуются обычная служба PostgreSQL, Python и venv. Команды для Windows/Linux, создание баз и настройка .env приведены отдельно в docs/INSTALL.md. При проверке использованы порты PostgreSQL 5433 и Uvicorn 8001; стандартные значения инструкции — 5432 и 8000.

Администратор PostgreSQL выполняет database.sql: создаются роль olympiad и две базы — рабочая и тестовая. После установки зависимостей из .env.example создаётся .env, генерируется JWT_SECRET длиной от 32 символов. Команда python init_db.py создаёт таблицы и по 50 учебных записей в пустой базе, повтор не меняет непустую.

Основные команды: python -m venv .venv; активировать окружение; python -m pip install -r requirements.txt; python init_db.py; python -m uvicorn app.main:app --reload. После запуска открыть / и /api/v1/docs, выполнить список, вход и профиль. Проверка процесса без запросов недостаточна.

Для тестов устанавливаются requirements-dev.txt и задаётся отдельная TEST_DATABASE_URL с окончанием _test. Команда python -m pytest создаёт тестовые таблицы, выполняет проверки и контролирует покрытие. Неверные адрес, порт или пароль базы исправляются в .env. Снимок запущенного API приведён в приложении 2 и руководстве установки.

Установка и подключение выполняются последовательно:

1. Установить Python и PostgreSQL; запустить службу PostgreSQL.

2. В корне проекта выполнить python -m venv .venv. Linux/macOS: source .venv/bin/activate; Windows PowerShell: .venv\Scripts\Activate.ps1.

3. Выполнить python -m pip install -r requirements.txt.

4. От администратора PostgreSQL выполнить database.sql. Linux: sudo -u postgres psql -v ON_ERROR_STOP=1 -f database.sql. Windows: в SQL Shell выполнить \i с полным путём к database.sql.

5. Скопировать .env.example в .env и указать адрес рабочей и отдельной тестовой базы.

6. Сгенерировать ключ: python -c "import secrets; print(secrets.token_hex(32))". Сохранить его в JWT_SECRET.

7. Выполнить python init_db.py, затем python -m uvicorn app.main:app --reload.

8. Открыть http://127.0.0.1:8000/api/v1/docs; войти через Authorize, получить профиль и список олимпиад.

9. Для тестов установить requirements-dev.txt, задать TEST_DATABASE_URL с окончанием _test, выполнить python -m pytest.

Пример .env для локальной учебной установки:
DATABASE_URL=postgresql+psycopg://olympiad:olympiad_local@localhost:5432/olympiads_simple

TEST_DATABASE_URL=postgresql+psycopg://olympiad:olympiad_local@localhost:5432/olympiads_simple_test
JWT_SECRET=<случайная строка от 32 символов>

На рисунке 7 показан фактический вывод команд подключения и проверки данных. Проверочная среда использует 5433/8001 из-за занятых обычных портов; в обычной инструкции используются 5432/8000. Вывод воспроизведён из реально выполненных команд, пароль подключения не показан.

Фактический вывод проверки установки

```
Проверка установки API: 23.10.2026 03:32:12 (Екатеринбург)
PostgreSQL установлен как обычная служба; Docker не используется.
> Проверить подключение и версию PostgreSQL
PostgreSQL 17.11 (Debian 17.11-0+deb13u1) on x86_64-pc-linux-gnu
Рабочая база проверки: olympiads_simple_delivery
SELECT 1 -> 1
> python init_db.py (повторная инициализация)
В базе уже есть пользователи. Данные не изменены.
> Проверить заполнение
users: 50 записей
olympiads: 50 записей
participations: 50 записей
attachments: 50 записей
> python -m uvicorn app.main:app --host 127.0.0.1 --port 8001
Рабочий запуск проверяется HTTP-запросами и снимком Swagger.
```

Рисунок 7 — Фактическая проверка установки и подключения PostgreSQL

Если подключение отклонено, проверить службу, host, port, роль и пароль. Если таблиц нет — выполнить init_db.py с адресом нужной базы. Если JWT_SECRET короче 32 символов — заменить ключ и перезапустить сервер. Если Swagger не загрузил оформление, проверить CDN; сами методы можно проверить через OpenAPI или HTTP-клиент. Непустая база не очищается и автоматически не дополняется.

Данные PostgreSQL и каталог storage хранятся отдельно от исходников. При переносе проекта нужно сохранить обе части; копирование только Python-файлов не переносит пользовательские документы. Повторный запуск не очищает рабочую базу.

3.6 Результаты тестирования

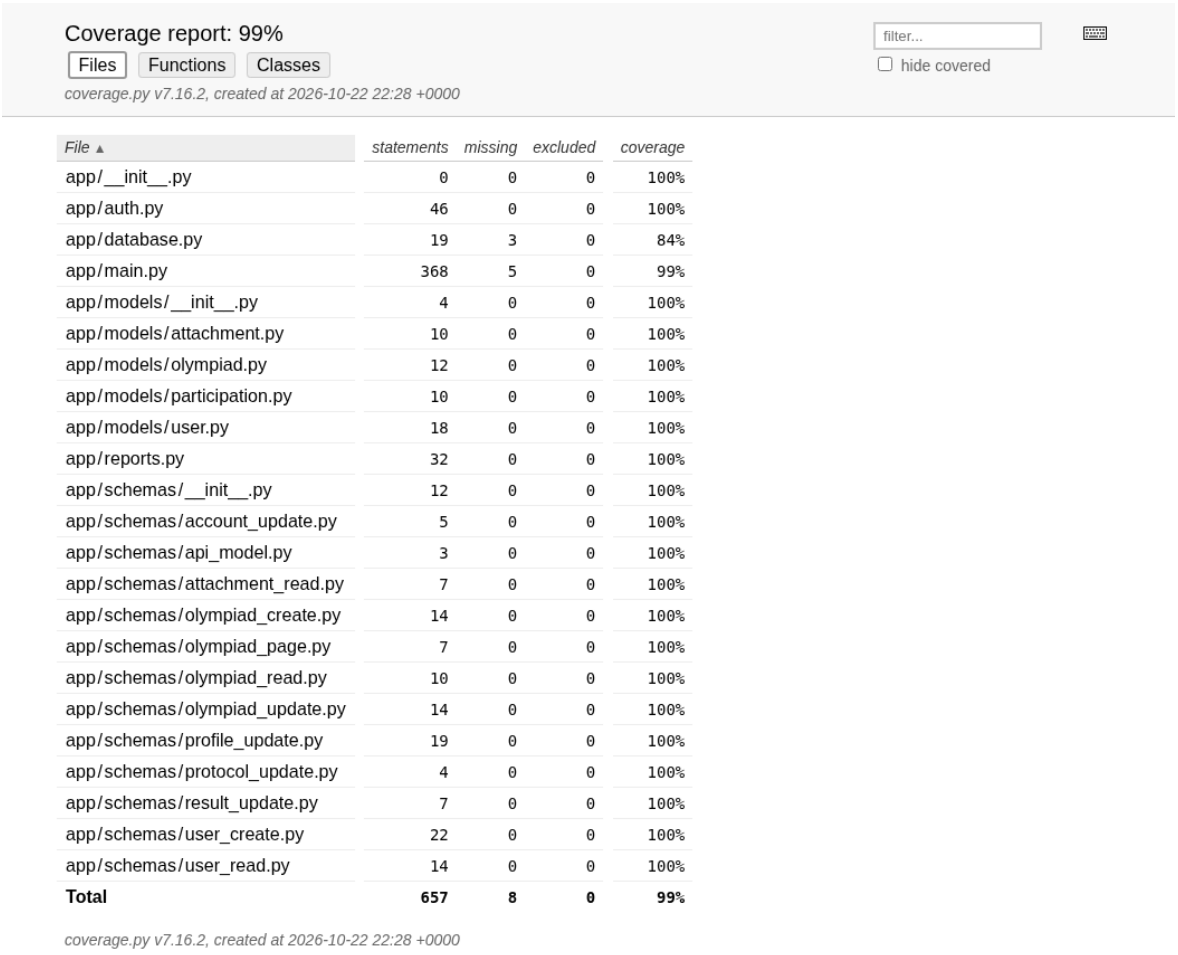
Повторный автоматический прогон 23.10.2026 03:27:48 по Екатеринбургу: 72 теста, успешно 72, ошибок и пропусков 0. Длительность — 16,93 с; время по XML — 16,862 с. Покрытие строк app — 98,78% (649 из 657), main.py с маршрутами и общими проверками — 98,64% (363 из 368). Требуемые 80% достигнуты.

Тесты проверили операции учётных записей, карточки, отбор и версии списка, роли, принадлежность преподавателя, заявки, результаты, переходы статусов, документы и экспорт. В бинарных отчётах проверено фактическое содержимое таблиц, а не только HTTP-код. В Excel отдельно проверено сохранение строки, начинающейся со знака формулы, в безопасном текстовом виде.

Через настоящий Uvicorn выполнены входы трёх ролей, обе версии документации, создание отдельной олимпиады, заявка, 95,5 балла, подготовка, публикация и четыре Word/Excel-выгрузки. До публикации гость получил 404, студент — null; после публикации результат открыт. Выполнен публичный GET из Swagger. Проверочные записи затем удалены из отдельной базы.

В pytest учтён пакет app, поэтому показатель покрытия не относится ко всем файлам репозитория. Наполнение дополнительно проверено отдельным тестом: все четыре таблицы, четыре вида документа и открытие 50 DOCX. Нагрузочные испытания, длительная устойчивость и пользовательские Django-страницы не проверялись.

Сопоставление с обязательными требованиями выявило отсутствие полного PUT карточки, замены бланка по ID, достаточного заполнения и тестов в первоначальной простой ветке. Они дополнены. Реестр наблюдавшихся проблем и закрытие приведены в приложении 1; подробный фактический отчёт — в docs/TEST_REPORT.md.



Coverage report: 99%

Files | Functions | Classes

filter... hide covered

coverage.py v7.16.2, created at 2026-10-22 22:28 +0000

File ▲	statements	missing	excluded	coverage
app/__init__.py	0	0	0	100%
app/auth.py	46	0	0	100%
app/database.py	19	3	0	84%
app/main.py	368	5	0	99%
app/models/__init__.py	4	0	0	100%
app/models/attachment.py	10	0	0	100%
app/models/olympiad.py	12	0	0	100%
app/models/participation.py	10	0	0	100%
app/models/user.py	18	0	0	100%
app/reports.py	32	0	0	100%
app/schemas/__init__.py	12	0	0	100%
app/schemas/account_update.py	5	0	0	100%
app/schemas/api_model.py	3	0	0	100%
app/schemas/attachment_read.py	7	0	0	100%
app/schemas/olympiad_create.py	14	0	0	100%
app/schemas/olympiad_page.py	7	0	0	100%
app/schemas/olympiad_read.py	10	0	0	100%
app/schemas/olympiad_update.py	14	0	0	100%
app/schemas/profile_update.py	19	0	0	100%
app/schemas/protocol_update.py	4	0	0	100%
app/schemas/result_update.py	7	0	0	100%
app/schemas/user_create.py	22	0	0	100%
app/schemas/user_read.py	14	0	0	100%
Total	**657**	**8**	**0**	**99%**

coverage.py v7.16.2, created at 2026-10-22 22:28 +0000

Рисунок 8 — Отчёт фактического автоматического прогона

Информационный заголовок отчёта: проект EMK-API-2026; версия simple-student после разделения классов; тип — версионный; дата повторной проверки — 23.10.2026; ФИО исполнителя при сдаче — _____. Проверены аккаунты, роли/JWT, CRUD, версии, заявки, результаты, протокол, файлы и Word/Excel. Падений и пропусков нет; повторный запуск после изменения структуры сохранил успешность набора. Многодневная стабильность не измерялась.

Для воспроизведения: установить requirements-dev.txt и выполнить python -m pytest. Дополнительные отчёты сохраняются параметрами --cov-report=html:reports/coverage --junitxml=reports/junit.xml. Исходные данные описаны в таблице 5, стратегия — в таблице 3, трассировка — в таблице 4. Общая успешность — 100%; неисполненные строки и границы покрытия указаны в TEST_REPORT.md.

4 ГЛОССАРИЙ

В таблице 13 приведены термины и сокращения, используемые в работе.

Таблица 13 — Термины и сокращения

Термин	**Определение**
АРМ	Автоматизированное рабочее место.
API	Программный интерфейс для взаимодействия приложений.
REST	Подход к организации HTTP-взаимодействия с ресурсами.
HTTP	Протокол запросов и ответов клиента и сервера.
JSON	Текстовый формат обмена структурированными данными.
JWT	Подписанный токен с набором утверждений и сроком действия.
ORM	Отображение объектов программы на таблицы базы данных.
БД / СУБД	База данных / система управления базами данных.
PK / FK	Первичный ключ / внешний ключ.
CRUD	Создание, чтение, обновление и удаление записей.
OpenAPI	Машиночитаемое описание операций HTTP API.

Продолжение таблицы 13

Термин	**Определение**
Swagger / ReDoc	Интерфейсы чтения и проверки OpenAPI.
PBKDF2	Функция получения хеша пароля с солью и повторными вычислениями.
Транзакция	Группа операций с подтверждением или откатом.
Пагинация	Получение списка частями заданного размера.
Фикстура	Подготовка и освобождение состояния теста.
Покрытие	Доля строк или ветвей кода, выполненных при проверках.
TDD	Разработка через цикл тест — реализация — рефакторинг.

5 СПИСОК ИСТОЧНИКОВ

1. Beaulieu A. Learning SQL. 3rd ed. O'Reilly Media, 2020.

2. Brewer E. CAP Twelve Years Later: How the "Rules" Have Changed // Computer. 2012. Vol. 45, no. 2. P. 23–29.

3. Chen P. P.-S. The Entity-Relationship Model—Toward a Unified View of Data // ACM Transactions on Database Systems. 1976. Vol. 1, no. 1. P. 9–36.

4. Codd E. F. A Relational Model of Data for Large Shared Data Banks // Communications of the ACM. 1970. Vol. 13, no. 6. P. 377–387.

5. Dijkstra E. W. The Humble Programmer // Communications of the ACM. 1972. Vol. 15, no. 10. P. 859–866.

6. FastAPI. Официальная документация: зависимости, безопасность, подприложения и тестирование. URL: https://fastapi.tiangolo.com/. Дата обращения к официальной документации и её исходным файлам: 23.10.2026.

7. Fenton N. E., Neil M. A Critique of Software Defect Prediction Models // IEEE Transactions on Software Engineering. 1999. Vol. 25, no. 5. P. 675–689.

8. Fowler M. Refactoring: Improving the Design of Existing Code. 2nd ed. Addison-Wesley, 2018.

9. Kleppmann M. Designing Data-Intensive Applications. O'Reilly Media, 2017.

10. Lampson B. W. Hints for Computer System Design // ACM SIGOPS Operating Systems Review. 1983. Vol. 17, no. 5. P. 33–48.

11. Martin R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.

12. Matthes E. Python Crash Course. 3rd ed. No Starch Press, 2023.

13. McCabe T. J. A Complexity Measure // IEEE Transactions on Software Engineering. 1976. Vol. SE-2, no. 4. P. 308–320.

14. Myers G. J., Sandler C., Badgett T. The Art of Software Testing. 3rd ed. Wiley, 2011.

15. Ntafos S. C. A Comparison of Some Structural Testing Strategies // IEEE Transactions on Software Engineering. 1988. Vol. 14, no. 6. P. 868–874.

16. Parnas D. L. On the Criteria To Be Used in Decomposing Systems into Modules // Communications of the ACM. 1972. Vol. 15, no. 12. P. 1053–1058.

17. Percival H. J. W. Test-Driven Development with Python. 2nd ed. O'Reilly Media, 2017.

18. Ramalho L. Fluent Python. 2nd ed. O'Reilly Media, 2022.

19. Saltzer J. H., Schroeder M. D. The Protection of Information in Computer Systems // Proceedings of the IEEE. 1975. Vol. 63, no. 9. P. 1278–1308.

20. Silberschatz A., Korth H. F., Sudarshan S. Database System Concepts. 7th ed. McGraw-Hill, 2019.

21. Sommerville I. Software Engineering. 10th ed. Pearson, 2016.

22. ГОСТ 2.105–95. Единая система конструкторской документации. Общие требования к текстовым документам. Применение по предоставленной методичке.

23. Методические рекомендации по выполнению курсового проекта ПМ.05 / сост. А. В. Грибова. Екатеринбург: ГАПОУ СО «ЕМК», 2026.

24. Общие правила оформления курсовой работы. Раздел 3. Предоставленный PDF, без выходных данных.

25. Техническое задание «Пеннер (1)». Создание и тестирование АРМ учёта участия студентов в олимпиадном движении. Предоставленный материал, 2026.

ЗАКЛЮЧЕНИЕ

В рамках первого этапа разработан REST-сервис для обработки сведений об участии студентов в олимпиадах и конкурсах профессионального мастерства. Определены четыре роли и модель PostgreSQL из четырёх связанных таблиц. Реализованы создание учётных записей, карточки мероприятий, заявки, результаты, документы и формирование Word/Excel. Обязательные таблицы тестирования и описание каждого класса и каждой фикстуры включены в записку; отдельные руководства сохранены в комплекте.

Публикация отделена от подготовки протокола. До неё гость и студент не получают незавершённые результаты. Преподаватель ограничен своими мероприятиями, администратор управляет учётными записями и публикует подготовленные итоги. Валидация и ограничения БД препятствуют некорректным датам, повторным заявкам и ряду ошибочных переходов.

Автоматические проверки подтвердили основные операции, ограничения доступа и форматы. После обязательных дополнений достигнут минимум данных по 50 записей и порог покрытия 80%. v2 отличается пагинацией и поиском. Структура остаётся простой: несколько серверных файлов, тесты и только необходимая документация.

Первый этап API реализован и проверен в учебном окружении. Перечень требований и связи с проверками приведены в отдельных документах тестирования. Неизвестные ФИО руководителя, точная группа, номер студенческого, обозначение и подписи оставлены для заполнения по реальным сведениям; вымышленные реквизиты не подставлены.

Приложение 1
Баг-репорт

В таблице 14 приведены наблюдавшиеся проблемы и результаты исправления.

Таблица 14 — Выявленные проблемы

ID	Проблема	Важность	Шаги и факт	Исправление / статус
B01	Нет полного PUT карточки	Средняя	PUT от admin в v1/v2: 405 вместо 200	Добавлен; обе версии проверены, закрыт
B02	Нет изменения бланка по ID	Высокая	PUT грамоты от ответственного: 405	Добавлен; ID сохранён, старый файл удалён, закрыт
B03	Недостаточный объём данных	Средняя	3 пользователя, 2 олимпиады, 1 заявка, 0 документов	По 50 записей, 50 DOCX, повтор проверен; закрыт
B04	Тестов нет в проекте	Средняя	В первоначальной ветке отсутствует tests/	Добавлены тесты, зависимости и порог покрытия; закрыт
B05	Классы объединены в два файла	Средняя	Сверка п. 2.8.3: в models.py и schemas.py более одного класса	Каждый класс в отдельном файле; 72 теста проходят; закрыт

B01–B05 устранены. Недостающие PUT сначала проверены падающими тестами, затем реализованы. Полный повторный прогон успешен; структурное требование подтверждено проверкой каждого Python-файла. Исправления API действуют для обеих версий.

В выполненном наборе блокирующих ошибок не обнаружено; границы проверки указаны в разделе 3.6.

Приложение 2
Примеры работы API

На рисунках 9–11 показан работающий API ветки simple-student: Swagger, фактический ответ GET и вторая версия. Ключ JWT и токены на снимках отсутствуют. Инструкции, запросы и статусы представлены отдельно в docs/API.md и docs/INSTALL.md; дополнительные демонстрационные файлы в комплект не включены.

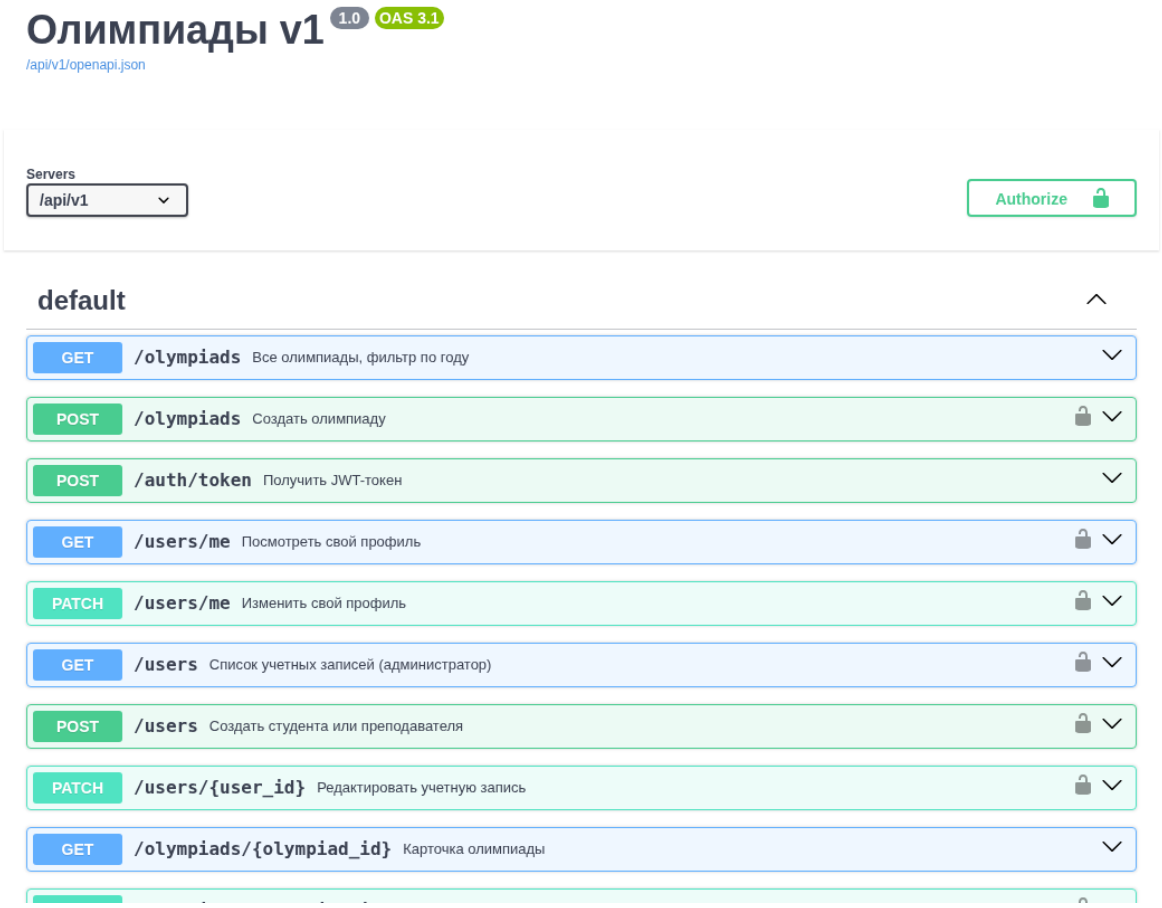


Рисунок 9 — Swagger первой версии работающего сервиса

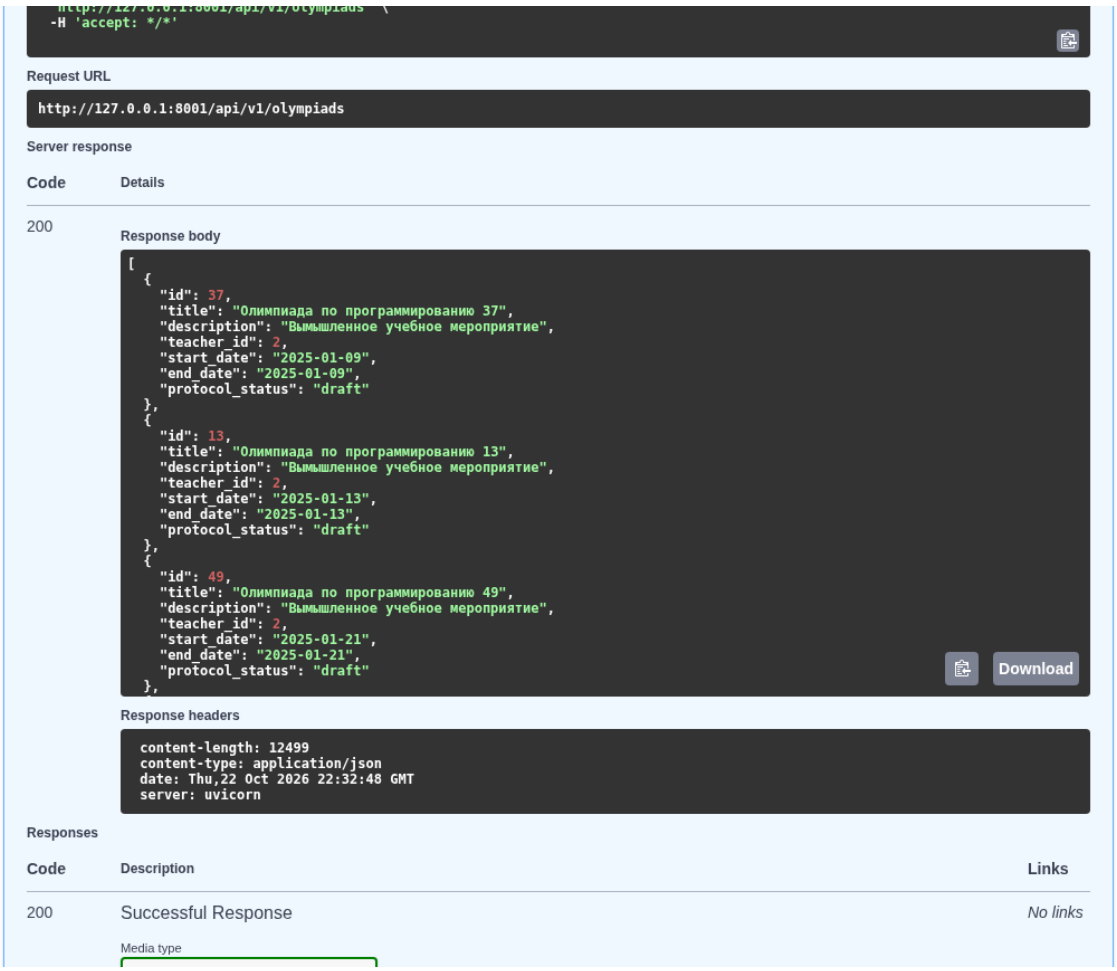


Рисунок 10 — Выполнение GET-запроса из Swagger

Публичные запросы выполняются без авторизации. Для изменяющих операций необходимо войти через Authorize. Код ответа, заголовки и тело позволяют проверить фактический результат, как в презентации requests.

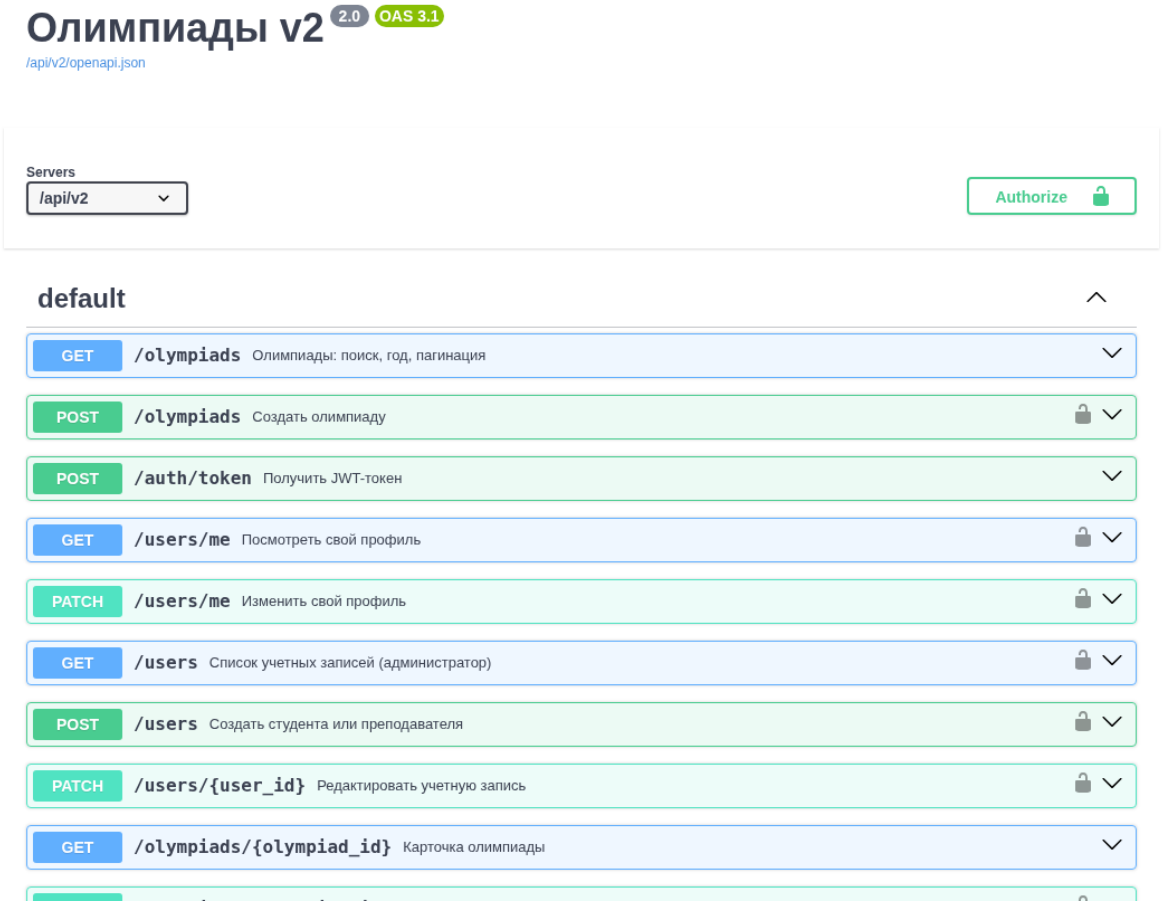


Рисунок 11 — Документация второй версии

У второй версии те же предметные операции и права. Список возвращает items, total, page, page_size; доступны поиск, отбор по году и проверка размера страницы. Контракт описан в docs/API.md.

Фактический HTTP-сценарий выполнен 22.10.2026 в отдельной базе с 50 олимпиадами. Пройдены вход, заявка, баллы, положение, подготовка, публикация и четыре отчёта. В таблице 15 приведена строка результата проверочного студента.

Таблица 15 — Строка проверочного опубликованного протокола

№	**ФИО**	**Баллы**	**Результат**
1	Студент Учебный Номер 1	95.5	Победитель

GET /api/v1/olympiads?year=2026
Authorization для публичного списка не требуется.
GET /api/v2/olympiads?page=1&page_size=10
POST /api/v1/auth/token (форма username и password)
GET /api/v1/users/me (Authorization: Bearer <JWT>)

Примеры создания студента, карточки, результата, публикации, загрузки и замены документа приведены в отдельном руководстве API.md. Установка и подключение к PostgreSQL — в INSTALL.md; подготовка, сценарии и результаты тестирования разделены на TEST_PLAN.md, TEST_CASES.md, TEST_REPORT.md.